Centro de Comercio y Servicios

Análisis y Desarrollo de Software

Instructor: Raúl Gómez

Asincronía del estado

El state puede ser actualizado de manera asíncrona. Es decir que state puede no ejecutarse inmediatamente, sino esperar a que React juzgue oportuno hacer las actualizaciones del estado. Esto podría resultar en una situación complicada de resolver, cuando el cálculo del nuevo valor del estado necesita basarse en el estado que haya anteriormente.

Asincronía del estado

EjemploAsync.jsx:

```
import { useState } from "react"

function EjemploAsync() {
  const [contador, setContador] = useState(0)

  const incrementar = () => {
    setContador(contador + 1)
    console.log("Valor después de setContador:", contador)
  }

  return (
    <div>
      <h2>Contador: {contador}</h2>
      <button onClick={incrementar}>Incrementar</button>
    </div>
  )
}

export default EjemploAsync
```

Asincronía del estado

App.jsx:

```
import './App.css'
import EjemploAsync from './components/EjemploAsync'

function App() {
  return (
    <>
      <div>
        <EjemploAsync />
      </div>
    </>
  )
}

export default App
```

Asincronía del estado

Ejecutar el proyecto

Contador: 1

Incrementar

Elements Console Sources Network

top ▾ Filter Default levels ▾ No Issues 2 hidden

```
react-dom_client.js?v=9d311806:20101
Download the React DevTools for a better development experience:
https://react.dev/link/react-devtools
Valor después de setContador: 0                    EjemploAsync.jsx:8
>
```

Asincronía del estado

En el código del método incrementar anterior, se basa en el estado actual del contador, para incrementarlo en una unidad. Sin embargo, si no se sabe realmente cuándo se va a ejecutar state. Por ello, podría ocurrir que ese incremento se realice a partir de un valor del estado que no era realmente el válido.

Asincronía del estado

EjemploAsync.jsx:

```
import { useState } from "react"

function EjemploAsync() {
  const [contador, setContador] = useState(0)

  const incrementar = () => {
    setContador(contador + 1)
    setContador(contador + 1)
    console.log("Valor después de setContador:", contador)
  }

  return (
    <div>
      <h2>Contador: {contador}</h2>
      <button onClick={incrementar}>Incrementar</button>
    </div>
  )
}

export default EjemploAsync
```

Asincronía del estado

Ejecutar el proyecto

Contador: 2

Incrementar

Elementos Consola Fuentes Red Rendimiento

top ▾ Filtrar Niveles predeterminados ▾

No hay problemas 2 ocultos

react-dom_client.js?v=9d311806:20101

Download the React DevTools for a better development experience: https://react.dev/link/react-devtools

Valor después de setContador: 0 EjemploAsync.jsx:9

Valor después de setContador: 1 EjemploAsync.jsx:9

>

Asincronía del estado

Para resolver esta situación en React se puede usar un mecanismo de manipulación del estado basado en una función.

En lugar de enviarle un objeto a setState() le estamos enviando una función. Esa función es capaz de recibir como parámetro el estado actual. Entonces, el nuevo estado se calcula en función del estado actual.

Asincronía del estado

EjemploAsync.jsx:

```
import { useState } from "react"

function EjemploAsync() {
  const [contador, setContador] = useState(0)

  const incrementar = () => {
    setContador((prev) => prev + 1)
    setContador((prev) => prev + 1)
    console.log("Valor después de setContador:", contador)
  }

  return (
    <div>
      <h2>Contador: {contador}</h2>
      <button onClick={incrementar}>Incrementar</button>
    </div>
  )
}

export default EjemploAsync
```

Asincronía del estado

Ejecutar el proyecto

Contador: 4

Incrementar

Elementos Consola Fuentes Red Rendimiento

top ▾ Filtrar Niveles predeterminados ▾

No hay problemas 2 ocultos

```
react-dom_client.js?v=9d311806:20101
Download the React DevTools for a better development experience:
https://react.dev/link/react-devtools
Valor después de setContador: 0                    EjemploAsync.jsx:9
Valor después de setContador: 2                    EjemploAsync.jsx:9
>
```

HttpClient

HttpClient es un servicio con métodos para realizar peticiones HTTP en aplicaciones React.

En la mayoría de casos, la comunicación entre aplicaciones Front-end y servicios Back-end se realiza mediante el protocolo HTTP. Para realizar este tipo de comunicación, las aplicaciones se apoyan en API como la interfaz XMLHttpRequest y la API fetch(), que son soportadas por la mayoría de navegadores web y permiten al Front-end realizar peticiones HTTP.

HttpClient

Se utiliza principalmente para consumir APIs (por ejemplo, una API creada con Django REST Framework o Node.js).

Peticiones HTTP

- Get
- Post
- Put
- Delete
- Patch
- Head
- jsonp

Operaciones Get y Post

Se utiliza el método get() para obtener un recurso del servidor y post() para crearlo.

HTTP Client

Instalar Axios

```
npm install axios
```

HTTP Client (GET)

Usuarios.jsx:

```
import axios from "axios";
import { useEffect, useState } from "react";

function Usuarios() {

  const [usuarios, setUsuarios] = useState([]);

  useEffect(() => {
    axios.get("https://jsonplaceholder.typicode.com/users")
      .then(res => {
        setUsuarios(res.data);
      })
      .catch(error => {
        console.log(error);
      });
  }, []);

  return (
    <div>
      <h2>Usuarios</h2>

      {usuarios.map(user => (
        <p key={user.id}>{user.name}</p>
      ))}

    </div>
  );
}

export default Usuarios;
```

HTTP Client (GET)

App.jsx:

```
import './App.css'
import Usuarios from './components/Usuarios'

function App() {
  return (
    <div>
      <Usuarios />
    </div>
  )
}

export default App
```

HTTP Client (GET)

Ejecutar el proyecto

Usuarios

Leanne Graham

Ervin Howell

Clementine Bauch

Patricia Lebsack

Chelsey Dietrich

Mrs. Dennis Schulist

Kurtis Weissnat

Nicholas Runolfsdottir V

Glenna Reichert

Clementina DuBuque

HTTP Client (POST)

FormUsuario.jsx:

```
import { useState } from "react";
import axios from "axios";

function FormUsuario(){

  const [nombre, setNombre] = useState("");

  const guardarUsuario = async () => {

    const respuesta = await axios.post(
      "https://jsonplaceholder.typicode.com/users",
      { name: nombre }
    );

    console.log(respuesta.data);
  };

  return(
    <div>

      <input
        type="text"
        value={nombre}
        onChange={(e)=>setNombre(e.target.value)}
      />

      <button onClick={guardarUsuario}>
        Guardar
      </button>

    </div>
  );
}

export default FormUsuario;
```

HTTP Client (POST)

App.jsx:

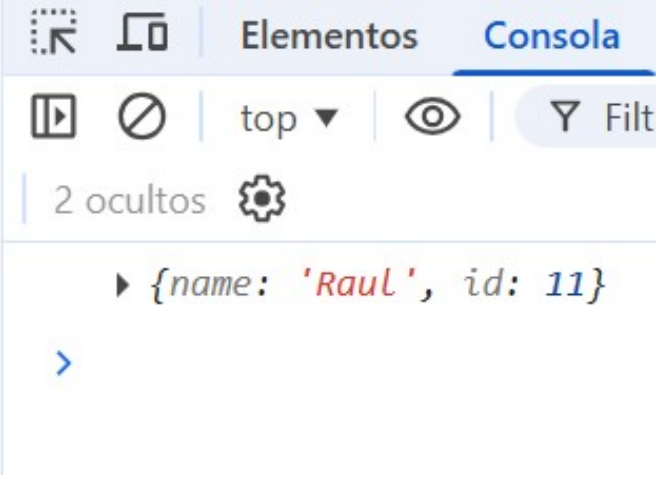
```
import './App.css'
import FormUsuario from './components/FormUsuario'

function App() {
  return (
    <div>
      <FormUsuario />
    </div>
  )
}

export default App
```

HTTP Client (POST)

Ejecutar el proyecto



Servicios

Separar las llamadas en servicios

Servicios

api.js:

```
import axios from "axios";

const API = axios.create({
  baseURL: "https://jsonplaceholder.typicode.com"
});

export const obtenerUsuarios = () => {
  return API.get("/users");
};
```

Servicios

Usuarios.jsx:

```
import { useEffect, useState } from "react";
import { obtenerUsuarios } from "../services/api";

function Usuarios() {

  const [usuarios, setUsuarios] = useState([]);

  useEffect(() => {
    cargarUsuarios();
  }, []);

  const cargarUsuarios = async () => {
    const res = await obtenerUsuarios();
    setUsuarios(res.data);
  };

  return (
    <div>
      <h2>Usuarios</h2>

      {usuarios.map(user => (
        <p key={user.id}>{user.name}</p>
      ))}

    </div>
  );
}

export default Usuarios;
```

Servicios

App.jsx:

```
import './App.css'
import Usuarios from './components/Usuarios'

function App() {
  return (
    <div>
      <Usuarios />
    </div>
  )
}

export default App
```

Servicios

Ejecutar el proyecto

Usuarios

Leanne Graham

Ervin Howell

Clementine Bauch

Patricia Lebsack

Chelsey Dietrich

Mrs. Dennis Schulist

Kurtis Weissnat

Nicholas Runolfsdottir V

Glenna Reichert

Clementina DuBuque

Parámetros HTTP

En el consumo de una API REST se pueden enviar parámetros HTTP (query params) en la URL para filtrar, buscar o paginar datos.

Los HTTP params se envían en la URL después de ?.

Parámetros HTTP

Ejemplo de URL:

```
https://api.com/usuarios?edad=30&ciudad=bogota
```

Parámetros HTTP

api.js:

```
import axios from "axios";

const API = axios.create({
  baseURL: "https://jsonplaceholder.typicode.com"
});

export const obtenerUsuarios = (params) => {
  return API.get("/users", {
    params: params
  });
};
```

Parámetros HTTP

BuscarUsuarios.jsx:

```
import { useState } from "react";
import { obtenerUsuarios } from "../services/api";

function BuscarUsuarios() {

  const [nombre, setNombre] = useState("");
  const [usuarios, setUsuarios] = useState([]);

  const buscar = async () => {

    const response = await obtenerUsuarios({
      username: nombre
    });

    setUsuarios(response.data);
  };

  return (
    <div>

      <h2>Buscar Usuario</h2>

      <input
        type="text"
        placeholder="Nombre usuario"
        value={nombre}
        onChange={(e) => setNombre(e.target.value)}
      />

      <button onClick={buscar}>
        Buscar
      </button>

      <ul>
        {usuarios.map(u => (
          <li key={u.id}>
            {u.name}
          </li>
        ))}
      </ul>

    </div>
  );
}

export default BuscarUsuarios;
```

Parámetros HTTP

App.jsx:

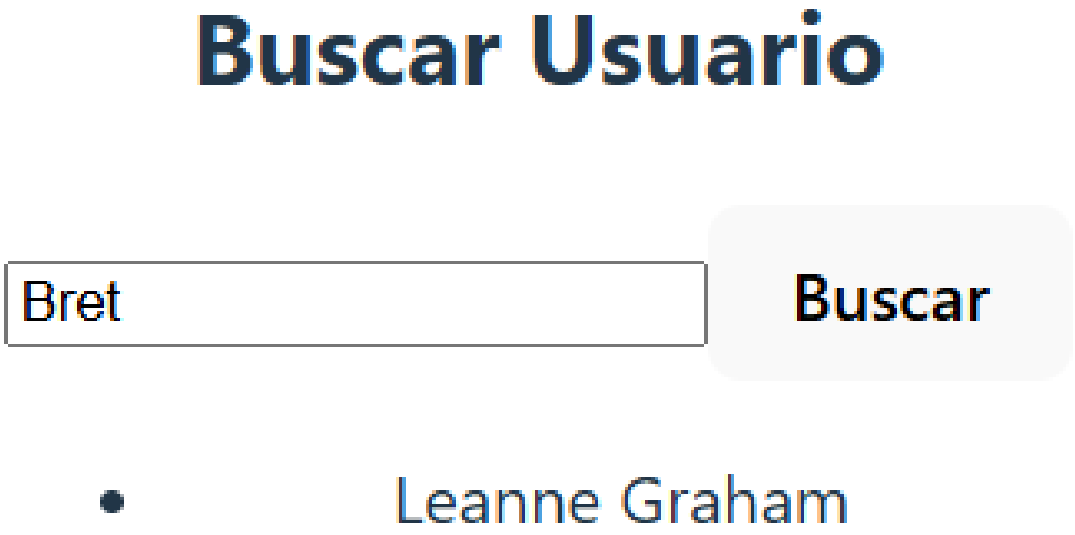
```
import './App.css'
import BuscarUsuarios from './components/BuscarUsuarios'

function App() {
  return (
    <div>
      <BuscarUsuarios />
    </div>
  )
}

export default App
```

Parámetros HTTP

Ejecutar el proyecto



Parámetros HTTP

Enviar múltiples parámetros

```
obtenerUsuarios({
  username: "Bret",
  id: 1
});
```

Operaciones put, patch y delete

Se utiliza put() y patch() para actualizar recursos del servidor de forma total o parcial, respectivamente, y delete() para eliminarlos.

Operaciones put, patch y delete

PUT(): para actualizar un recurso del servidor.

PATCH(): para actualizar, de forma parcial, un recurso del servidor. Lo utilizaríamos cuando, por ejemplo, solamente necesitáramos actualizar una propiedad del recurso.

DELETE(): para eliminar un recurso del servidor.

Operaciones put, patch y delete

api.js:

```
import axios from "axios";

const API = axios.create({
  baseURL: "https://jsonplaceholder.typicode.com"
});

export const obtenerUsuarios = () => API.get("/users");

export const crearUsuario = (usuario) =>
  API.post("/users", usuario);

export const actualizarUsuario = (id, usuario) =>
  API.put(`/users/${id}`, usuario);

export const eliminarUsuario = (id) =>
  API.delete(`/users/${id}`);
```

Operaciones put, patch y delete

UsuarioList.jsx:

```
import { useEffect, useState } from "react";
import { obtenerUsuarios, eliminarUsuario } from "../services/api";

function UsuarioList({ onEditar }) {

  const [usuarios, setUsuarios] = useState([]);

  useEffect(() => {
    cargarUsuarios();
  }, []);

  const cargarUsuarios = async () => {
    const res = await obtenerUsuarios();
    setUsuarios(res.data);
  };

  const borrar = async (id) => {
    await eliminarUsuario(id);
    cargarUsuarios();
  };

  return (
    <div>

      <h2>Lista de Usuarios</h2>

      {usuarios.map(user => (
        <div key={user.id}>

          {user.name}

          <button onClick={() => onEditar(user)}>
            Editar
          </button>

          <button onClick={() => borrar(user.id)}>
            Eliminar
          </button>

        </div>
      ))}

    </div>
  );
}

export default UsuarioList;
```

Operaciones put, patch y delete

UsuarioForm.jsx:

```
import { useState, useEffect } from "react";
import { crearUsuario, actualizarUsuario } from "../services/api";

function UsuarioForm({ usuarioEditar, recargar }) {

  const [nombre, setNombre] = useState("");

  useEffect(() => {
    if (usuarioEditar) {
      setNombre(usuarioEditar.name);
    }
  }, [usuarioEditar]);

  const guardar = async () => {

    if (usuarioEditar) {

      await actualizarUsuario(usuarioEditar.id, {
        name: nombre
      });

    } else {

      await crearUsuario({
        name: nombre
      });

    }

    setNombre("");
    recargar();
  };

  return (
    <div>

      <h2>Formulario Usuario</h2>

      <input
        type="text"
        placeholder="Nombre"
        value={nombre}
        onChange={(e) => setNombre(e.target.value)}
      />

      <button onClick={guardar}>
        Guardar
      </button>

    </div>
  );
}

export default UsuarioForm;
```

Operaciones put, patch y delete

App.jsx:

```
import { useState } from "react";
import UsuarioForm from "./components/UsuarioForm";
import UsuarioList from "./components/UsuarioList";

function App() {

  const [usuarioEditar, setUsuarioEditar] = useState(null);
  const [actualizar, setActualizar] = useState(false);

  const recargar = () => {
    setActualizar(!actualizar);
    setUsuarioEditar(null);
  };

  return (

    <div>

      <h1>CRUD Usuarios</h1>

      <UsuarioForm
        usuarioEditar={usuarioEditar}
        recargar={recargar}
      />

      <UsuarioList
        onEditar={setUsuarioEditar}
        actualizar={actualizar}
      />

    </div>

  );
}

export default App;
```

Operaciones put, patch y delete

Ejecutar el proyecto

Ejercicios prácticos

Ejercicio 1

Mostrar un chiste aleatorio.

Pasos:

Llamar a https://api.chucknorris.io/jokes/random.

Mostrar el resultado (value) en un componente.

Agregar un botón "Otro chiste" para recargar uno nuevo.

Ejercicios prácticos

Ejercicio 2

Obtener y mostrar todas las categorías disponibles.

Pasos:

Consumir https://api.chucknorris.io/jokes/categories.

Mostrar las categorías en una lista.

Al hacer clic en una categoría, llamar a un método que obtenga un chiste de esa categoría (/jokes/random?category=...).

Ejercicios prácticos

Ejercicio 3

Buscar chistes por texto para mostrar múltiples chistes.

Pasos:

Crear un campo de texto (input) para ingresar la búsqueda.

Al hacer clic en "Buscar", llamar a: https://api.chucknorris.io/jokes/search?query=algo

Mostrar los resultados (result[] del objeto devuelto) en una lista.

Ejercicios prácticos

Ejercicio 4

Mostrar íconos de los chistes.

Pasos:

En cada tarjeta o fila de resultado, mostrar la imagen de:

{{ joke.icon_url }}

Mostrar también el value del chiste.

Ejercicios prácticos

Ejercicio 5

Buscar un cóctel por nombre y mostrar sus detalles.

Tareas:

Crear un formulario o campo de texto con botón “Buscar”.

Nombre del cóctel (strDrink)

Imagen (strDrinkThumb)

Instrucciones (strInstructions)

Lista de ingredientes (strIngredient1, strIngredient2, …)

https://www.thecocktaildb.com/api/json/v1/1/search.php?s=margarita

Ejercicios prácticos

Ejercicio 6

Mostrar cócteles según su categoría (Ej: "Ordinary Drink", "Cocktail", etc.)

https://www.thecocktaildb.com/api/json/v1/1/list.php?c=list

Mostrar las categorías en un select.

Al seleccionar una, usar:

https://www.thecocktaildb.com/api/json/v1/1/filter.php?c=Cocktail

Mostrar lista con nombre + imagen.

Ejercicios prácticos

Ejercicio 7

Al hacer clic en un cóctel de la lista, ver más detalles.

Pasos:

Capturar el ID del cóctel (idDrink) al hacer clic.

Llamar:

https://www.thecocktaildb.com/api/json/v1/1/lookup.php?i=11007

Mostrar datos completos en un componente aparte o en una tarjeta expandida.

Ejercicios prácticos

Ejercicio 8

Mostrar cócteles que contengan un ingrediente específico.

Pasos:

Usar un input para ingresar un ingrediente (ej: "Gin").

https://www.thecocktaildb.com/api/json/v1/1/filter.php?i=Gin

Mostrar nombre + imagen de cada cóctel resultante.

Ejercicios prácticos

Ejercicio 9

Mostrar un cóctel aleatorio al presionar un botón.

Pasos:

Al hacer clic en "Sorpresa", consumir:

https://www.thecocktaildb.com/api/json/v1/1/random.php

Ejercicios prácticos

Ejercicio 10

Mostrar cócteles filtrados por tipo: "Alcoholic", "Non_Alcoholic", "Optional_alcohol".

Pasos:

Mostrar un select con esas tres opciones.

Consumir:

https://www.thecocktaildb.com/api/json/v1/1/filter.php?a=Non_Alcoholic

Mostrar resultado con nombre + imagen.

Ejercicios prácticos

Ejercicio 11

Usar Routing con detalles de cócteles.

Pasos:

Componente ListadoComponent: muestra lista de cócteles.

Componente DetalleComponent: muestra detalles según id recibido por ruta.

Leer el ID y llamar a la API de detalles.

https://www.thecocktaildb.com/api.php

Ejercicios prácticos

Ejercicio 12

Obtener y mostrar una lista de personajes con nombre e imagen.

Pasos:

Llamar a:

https://rickandmortyapi.com/api/character

Mostrar name e image de cada personaje en tarjetas.

Ejercicios prácticos

Ejercicio 13

Permitir buscar personajes por nombre.

Pasos:

Agregar un input para ingresar texto.

Hacer una búsqueda:

https://rickandmortyapi.com/api/character/?name=rick

Ejercicios prácticos

Ejercicio 14

Al hacer clic en un personaje, mostrar su detalle completo.

Pasos:

Al seleccionar un personaje, redirigir a /personaje/:id.

Obtener el ID.

Llamar a:

https://rickandmortyapi.com/api/character/1

Mostrar nombre, estado, especie, género, origen, ubicación, e imagen.

Ejercicios prácticos

Ejercicio 15

Mostrar todos los episodios y permitir ver qué personajes aparecen en cada uno.

Pasos:

Consumir:

https://rickandmortyapi.com/api/episode

Mostrar name y episode (ej: S01E01).

Al hacer clic, llamar a cada URL de personaje en el array characters[].

Ejercicios prácticos

Ejercicio 16

Crear filtros por los atributos disponibles.

Pasos:

Estado: alive, dead, unknown

Género: male, female, genderless, unknown

Consumir:

https://rickandmortyapi.com/api/character/?status=alive&gender=male

Ejercicios prácticos

Ejercicio 17

Mostrar personajes en una tabla con columnas: nombre, especie, origen, ubicación.

https://rickandmortyapi.com/

Ejercicios prácticos

Ejercicio 18

Usar Router para separar vistas:

/personajes → lista

/personajes/:id → detalle individual

/episodios → lista de episodios

/episodios/:id → detalle de episodio con personajes

https://rickandmortyapi.com/

Ejercicios prácticos

Ejercicio 19

Mostrar el número total de personajes, episodios y ubicaciones

Llamar a:

/character

/episode

/location

Acceder a info.count de cada uno.

Mostrarlos como tarjetas.

https://rickandmortyapi.com/

Ejercicios prácticos

Ejercicio 20

Mostrar un personaje aleatorio al pulsar un botón.

Pasos:

Obtener la cantidad total de personajes (info.count).

Generar un ID aleatorio entre 1 y count.

Llamar a:

https://rickandmortyapi.com/api/character/{id}

Mostrar el personaje aleatorio.

Ejercicios prácticos

Ejercicio 21

Crear filtros por los atributos disponibles.

Tareas:

Usar select o radio para elegir valores:

Estado: alive, dead, unknown

Género: male, female, genderless, unknown

Consumir:

https://rickandmortyapi.com/api/character/?status=alive&gender=male